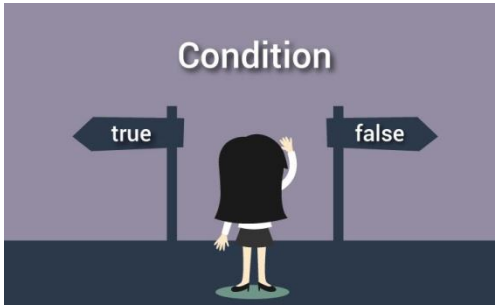
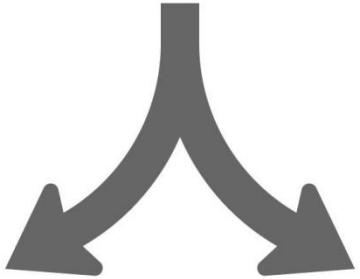
C Program Structure: If else and Looping

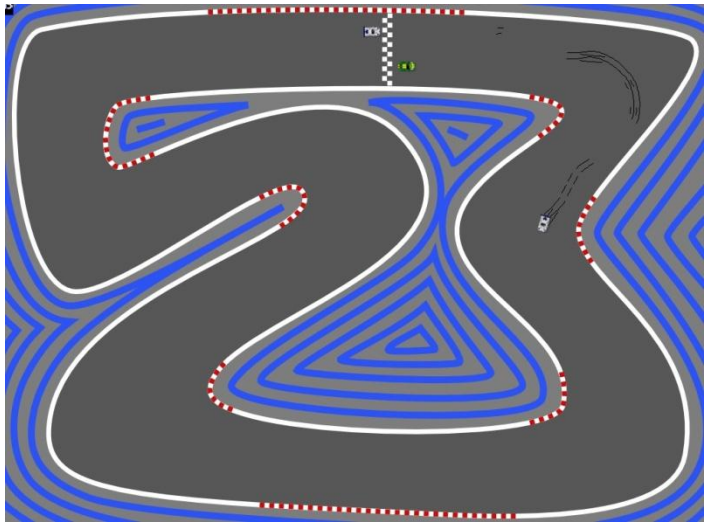
Control Statements

- Branching





- Looping



Control Statements

- Flow of control so far
  - Top to bottom

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  int n;
  double m;

  scanf(“%d”,&n);
  m = log(n);        // natural log
  printf("%f\n", m);

  return 0;
}
```

Branching: Use Case

- What happens when n is negative?

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  int n;
  double m;

  scanf(“%d”,&n);
  m = log(n);        // natural log
  printf("%f\n", m);

  return 0;
}
```

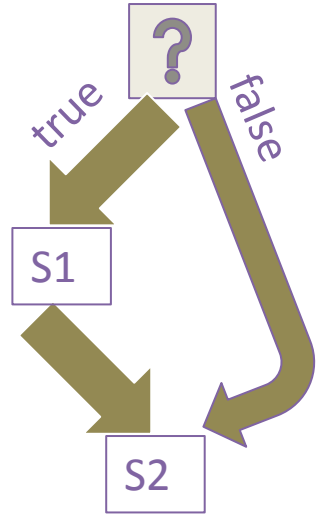
Branching Statements in C

- 3 types of conditional statements in C
  - if (cond) action
  - if (cond) action
    else some-other-action
  - switch-case
- Each action is a sequence of one or more statements!

If Statement

- General form of the if statement

```
if (expression)
        statement S1
statement S2
```

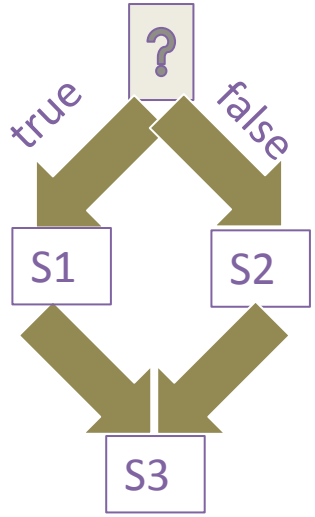


- Execution of if statement
  - First the expression is evaluated.
  - If it evaluates to a non-zero value, then S1 is executed and then control (program counter) moves to the statement S2.
  - If expression evaluates to 0, then S2 is executed.

if-else Statement

- General form of the if-else statement

```
if (expression)
        statement S1
else
        statement S2
statement S3
```



- Execution of if-else statement
  - First the expression is evaluated.
  - If it evaluates to a non-zero value, then S1 is executed and then control (program counter) moves to S3.
  - If expression evaluates to 0, then S2 is executed and then control moves to S3.
  - S1/S2 can be **block** of statements!

Solving the Use Case with if-else

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  int n;
  double m;
  printf(“Please enter a positive number: ”);
  scanf(“%d”,&n);
  if (n>0)
    m = log(n);          // natural log
    printf("%f\n", m);
  else
    printf(“Why can’t you follow instructions?");

return 0;
}
```

Something’s not quite right in this code

Compound Statements

- A block of code containing zero or more statements
- Contained between { and }
- Format:

```
{
  Local Declarations

  Statements
}
```

Getting it Right

```
#include <stdio.h>
#include <stdlib.h>

int main() {
  int n;
  double m;
  printf(“Please enter a positive number: ”);
  scanf(“%d”,&n);
  if (n>0){
    m = log(n);          // natural log
    printf("%f\n", m);
  }
  else
    printf(“Why can’t you follow instructions?");
  return 0;
}
```

if-else Statement

- Read two integers and print the min.

```
# include <stdio.h>
int main() {
        int x, y;
        scanf("%d%d",
&x,&y);
    if (x < y) {
        printf("%d", x);
    } else {
        printf("%d", y);
    }
    return 0;
}
```

1. Check if x is less than y.
2. If so, print x
3. Otherwise, print y.

Nested if-else

```
#include <stdio.h>
#include <stdlib.h>

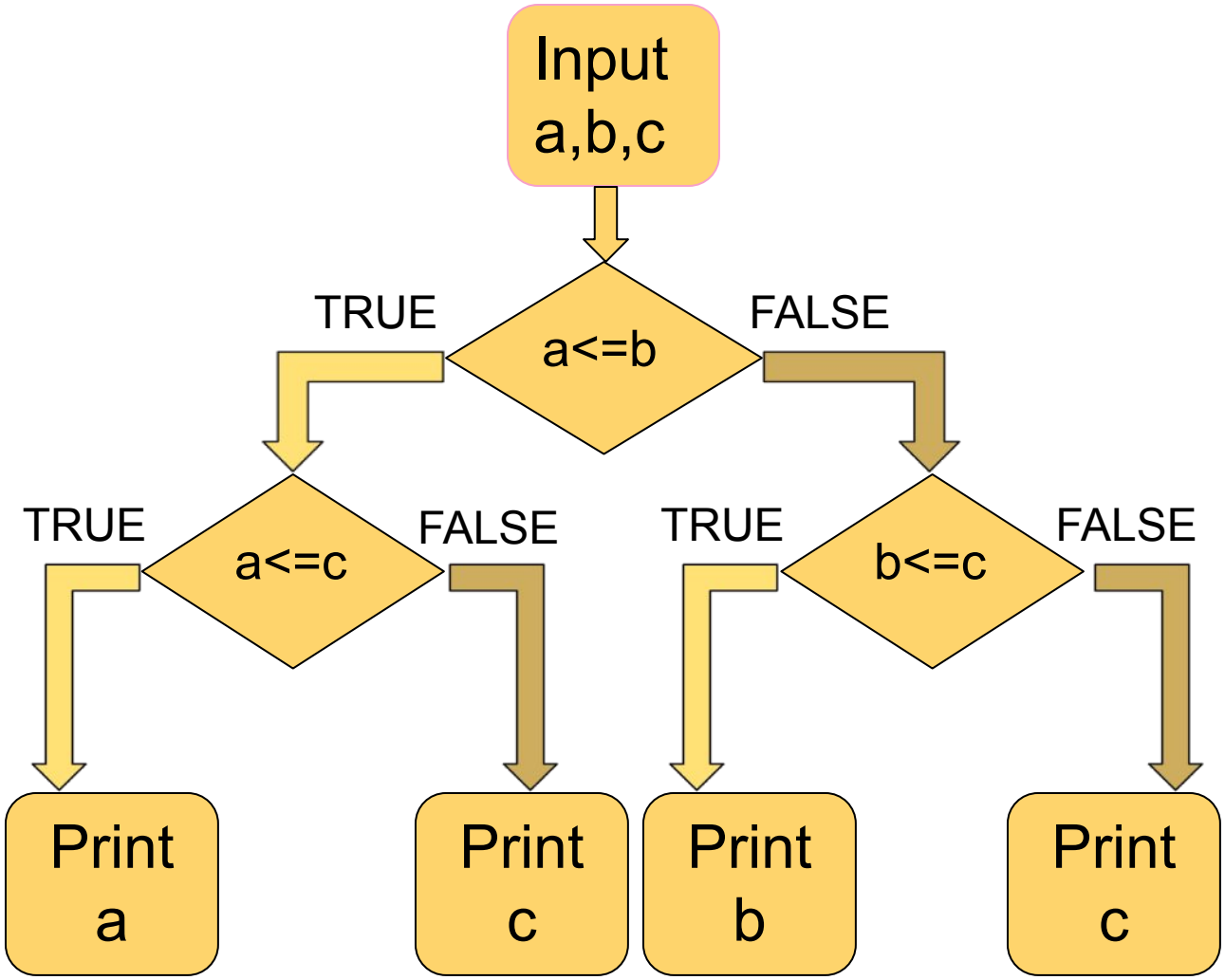
int main() {
    int c,d;
    printf(“Do you like C? (0/1) ”);
    scanf(“%d”,&c);
    if(c){
        printf(“Do you really like C? (0/1) ”);
        scanf(“%d”,&d);
        if(d)
            printf(“You really like C!!”);
        else
            printf(“Not really”);
    }
    else{
         printf(“Ohh, You hate C!!”);
    }


    return 0;
}
```

Nested if-else

- If e1 s1 else if e2 s2
- If e1 s1 else if e2 s2 else s3
- If e1 if e2 s1 else s2 else s3
- If e1 if e2 s1 else s2
- Rule of thumb: *else* clause looks for the closest previous *if* without an *else*

Finding min of 3 numbers



Switch-Case Statement

- Multi-way decision
- Checks whether an expression matches one out of a number of constant integer values
- Execution *branches* based on the match found

Printing the day

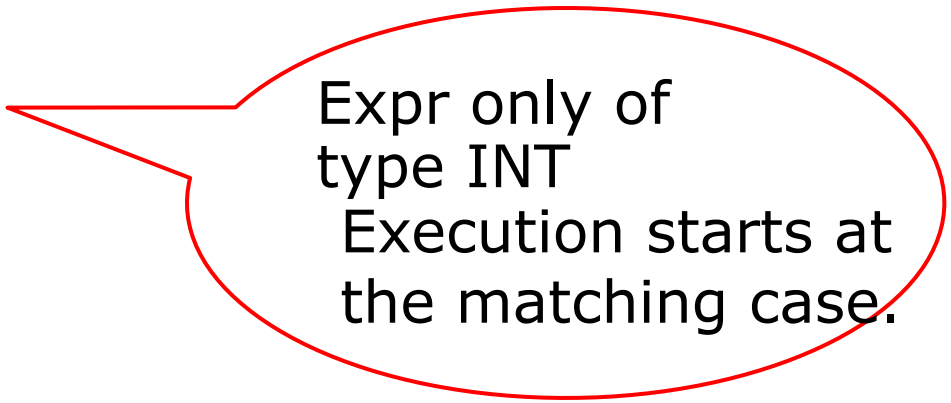
```
switch (day) {
case 1: printf(“Sunday”); break;
case 2: printf (“Monday”); break;
case 3: printf (“Tuesday”); break;
case 4: printf (“Wednesday”); break;
case 5: printf (“Thursday”); break;
case 6: printf (“Friday”); break;
case 7: printf (“Saturday”); break;
default: printf (“ Illegal day %d”, day);
}
```

The *break* Statement

- Syntax –
  - break;
- Used to break out of current code branch
  - Not to end the program

General Form of switch-case

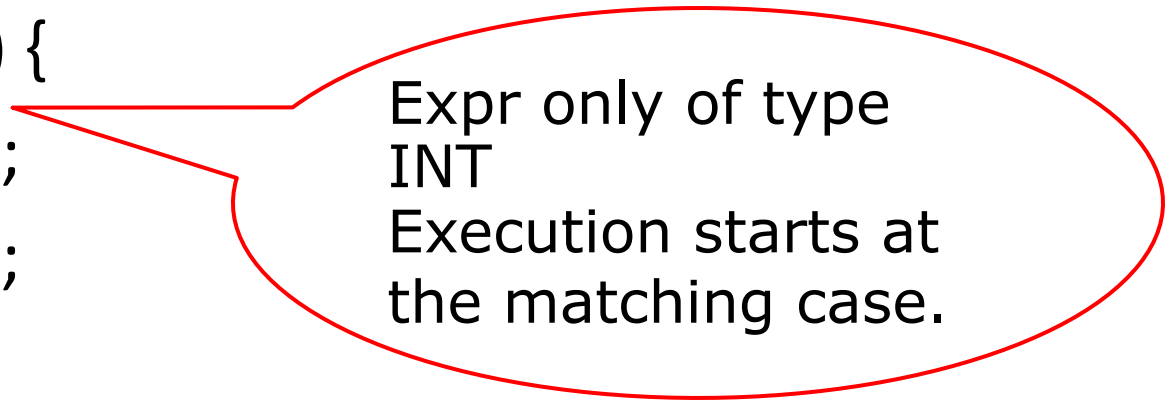
```
switch (selector-expr) {
case label1: s1; break;
case label2: s2; break;
...
case labelN: sN; break;
default : sD;
}
```



- **default** is optional. (= *remaining cases*)
- The location of **default** does not matter.
- The statements following a case label are executed one after other until a **break** is encountered (**Fall Through**)

General Form of switch-case

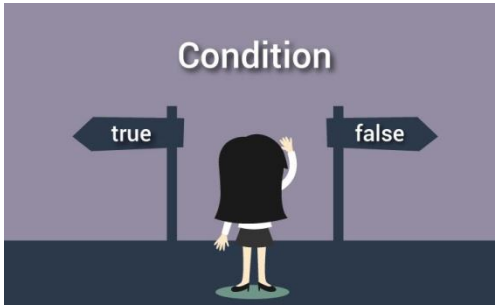
```
switch (selector-expr) {
case label1: s1; break;
case label2: s2; break;
...
case labelN: sN; break;
default : sD;
}
```

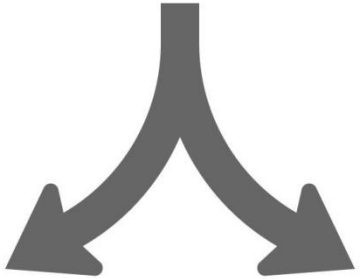


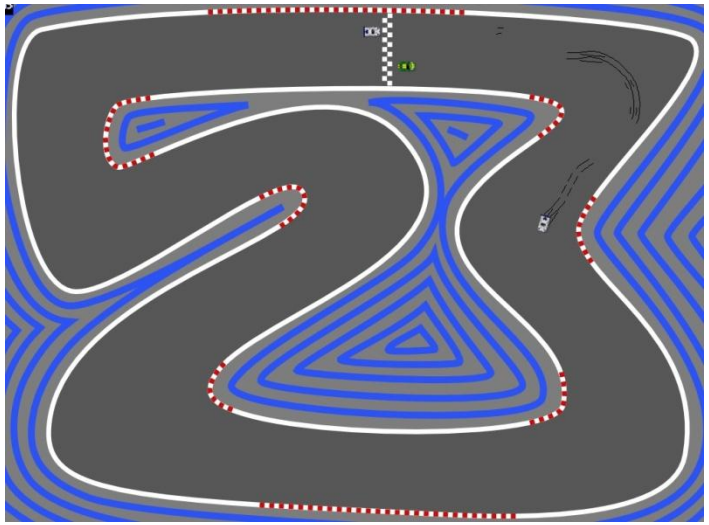
- **default** is optional. (= *remaining cases*)
- The location of **default** does not matter.
- The statements following a case label are executed one after other until a **break** is encountered (**Fall Through**)

Control Statements

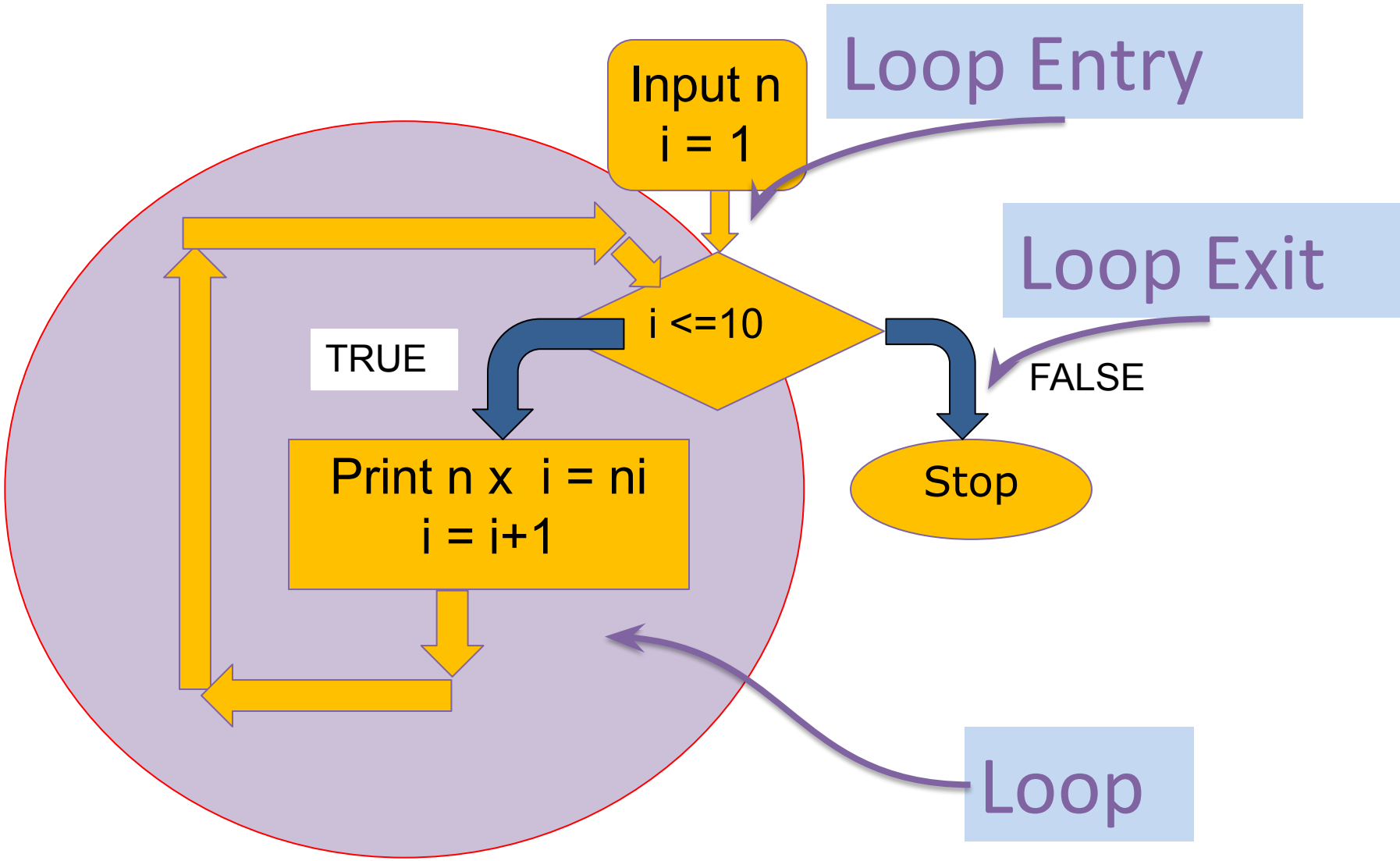
- Branching ✔
- Looping





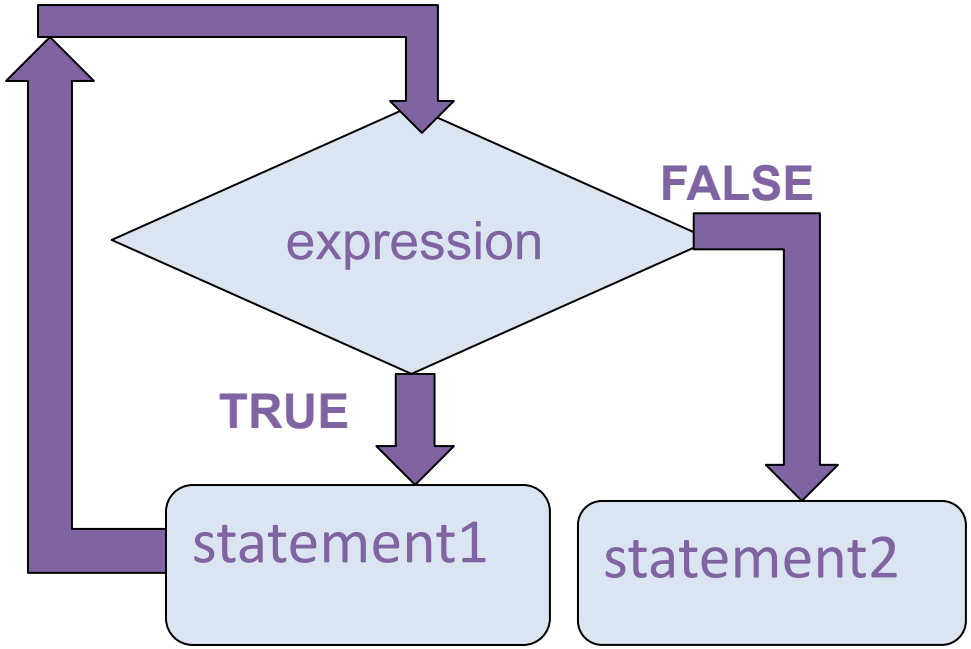


Printing Multiplication Table



While Statement

```
while (expression)
    statement1;
statement2;
```



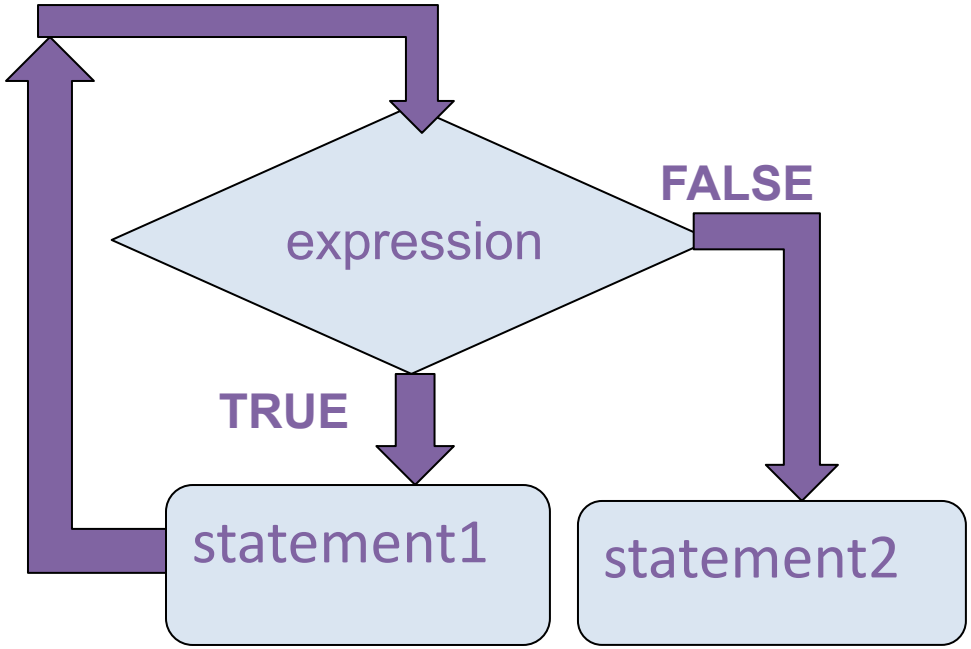
Read in English as:

As long as expression is TRUE execute statement1.

when expression becomes FALSE execute statement 2.

While Statement

```
while (expression)
    statement1;
statement2;
```



1. Evaluate expression
2. If TRUE then
   a) execute statement1
   b) goto step 1.
3. If FALSE then execute statement2.

Example

1. Read a sequence of integers from the terminal until -1 is read
2. Output sum of numbers read, not including the -1

First, let us write the loop, then add code for sum

```
int a;
scanf("%d", &a);              /* read into a */
while ( a !=  -1) {
    scanf("%d", &a);    /* read into a inside loop*/
}
```

Terminology

- Iteration: Each run of the loop is called an iteration.
  - In example, the loop runs for 3 iterations, corresponding to inputs 4, 15 and -5.
  - For input -1, the loop is exited, so there is no iteration for input -1.
- 3 components of a while loop
  - Initialization
    - first reading of a in example
  - Condition (evaluates to a Boolean value)
    - a != -1
  - Update
    - another reading of a

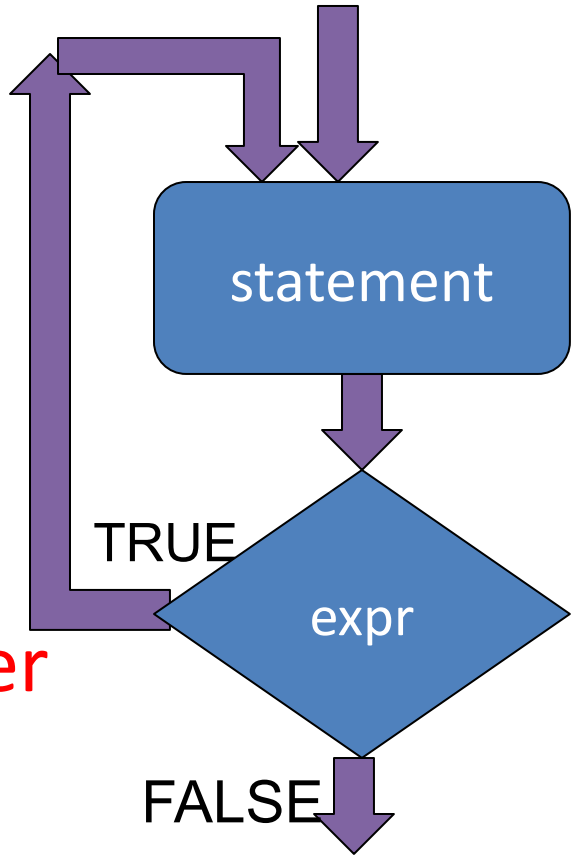
do-while Loops

- do-while statement is a variant of while.

General form:

```
do
    statement
while (expr);
```

- Execution:
  1. First execute statement.
  2. Then evaluate expr.
  3. If expr is TRUE then go to step 1.
  4. If expr is FALSE then break from loop
- Continuation of loop is tested after the statement.



Comparing while and do-while

- In a while loop the body of the loop may not get executed even once, whereas, in a do-while loop the body of the loop gets executed at least once.
- In the do-while loop structure, there is a semicolon after the condition of the loop.
- Rest is similar to a while loop.

Comparative Example

- Problem: Read integers and output each integer until -1 is seen (include -1 in output).
- The program fragments using while and do-while.

Using do-while

```
int a; /*current int*/

do {
   scanf("%d", &a);
   printf("%d\n", a);
} while (a != -1);
```

Using while

```
int a;/*current int*/

scanf("%d",&a);
while (a != -1) {
   printf("%d\n", a);
   scanf("%d", &a);
}
printf("%d\n", a);
```

For loop in C

- General form

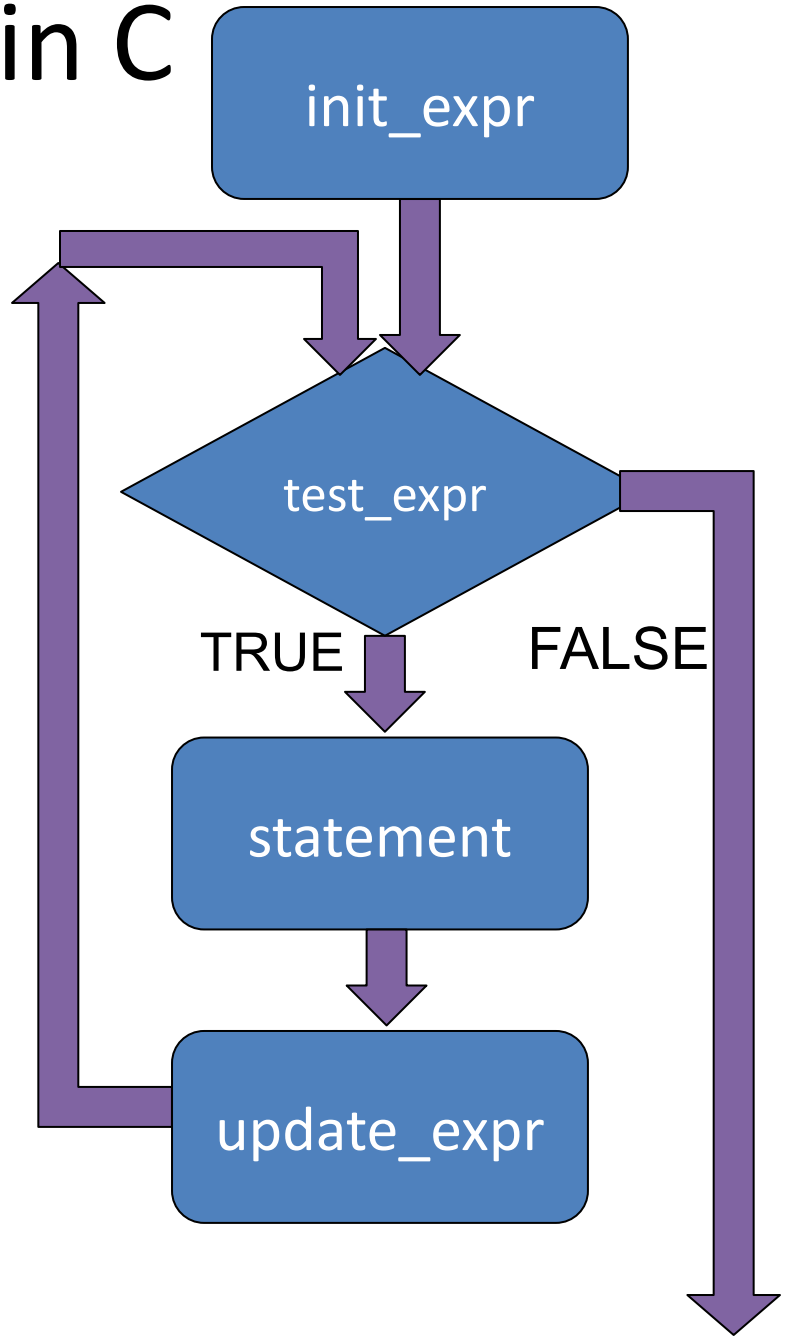
```
for (init_expr; test_expr; update_expr)
    statement;
```

- init_expr is the initialization expression.
- update_expr is the update expression.
- test_expr is the expression that evaluates to either TRUE (non-zero) or FALSE (zero).
- statement is the work to repeat (can be multiple statements in {…})

For Loop in C

```
for (init_expr; test_expr; update_expr)
      statement;
```

1. First evaluate **init_expr**;
2. Evaluate **test_expr**;
3. If **test_expr** is TRUE then
   a) execute **statement**;
   b) execute **update_expr**;
   c) go to Step 2.
4. if **test_expr** is FALSE then break from the loop



For Loop

- Print the sum of the reciprocals of the first 100 natural numbers.

```
int i; // counter from 1..100
float rsum = 0.0;// the sum

// the for loop
for ( i=1; i<=100; i=i+1 ) {
    rsum = rsum +  (1.0/i);
}
printf("sum is %f ", rsum);
```

for Loop in Terms of while Loop

```
for (init_expr; test_expr; update_expr)
    statement;
```

- Execution is (almost) equivalent to

```
init_expr;
while (test_expr) {
    statement;
    update_expr;
}
```

- Almost? Exception if there is a continue; inside statement– this will be covered later.
- Both are (almost) equivalent in power.
- Which loop structure to use, depends on the convenience of the programmer.

Nested Loops



- Loop with in a loop
- Many iterations of inner loop One iteration of outer loop





Example

- Write a program that displays the following pattern

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25
6	12	18	24	30
7	14	21	28	35
8	16	24	32	40

integers are printed in 5 columns each

```
#include<stdio.h>
int main(){
    int i, j;

    for (i=1; i<=8; i=i+1) {
        for (j=1; j<=5; j=j+1) {
            printf("%4d", i*j); // Displaying i, 2i, …, 5i
        }
        printf("\n"); // Move to the next line
    }

    return 0;
}
```

Done in lab

Displaying a Pattern

```
#include <stdio.h>
int main(){
  int i,j;
  for (i=1; i<=5; i=i+1){
    for (j=i; j<2*i; j=j+1){
      printf("%d ",j);
    }
    printf("\n");
  }
  return 0;
}
```

- Output?

1

2 3

3 4 5

4 5 6 7

5 6 7 8 9

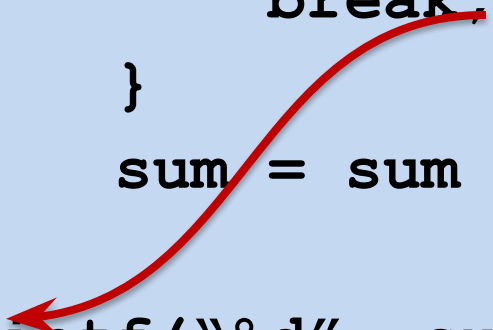
Back to Break



- Used for exiting a loop forcefully
- Example Program:

Read 100 integer inputs from a user. Print the sum of inputs until a negative input is found (Excluding the negative number) or all 100 inputs are exhausted.

```
int value;
int sum = 0;
int i;
for (i = 0; i < 100; i++) {
     scanf("%d", &value);
     if (value < 0) {
          //-ve number: no need to go
          // around the loop any more!!
          break;
     }
     sum = sum + value;
}
printf("%d", sum);
```



When to Break?

- Use of break sometimes can simplify exit condition from loop.
- However, it can make the code a bit harder to read and understand.
- Tip: if the loop terminates in at least two ways which are sufficiently different and requires substantially different processing then consider the use of termination via break for one of them.

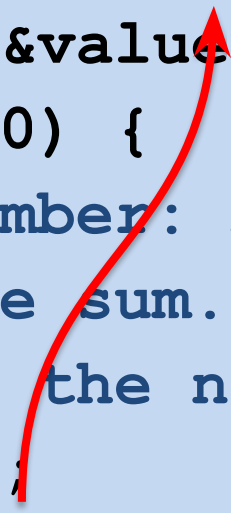
Continue

- Used for skipping an iteration of a loop
- The loop is NOT exited.
- Example Program:

Read 100 integer inputs from a user. Print the sum of only positive inputs.



```
int sum = 0;
int i, value;
for (i = 0; i < 100; i++) {
     scanf("%d", &value);
     if (value < 0) {
          //-ve number: no need to add it
          // to the sum. Go ahead and
          // check the next input.
          continue;
     }
     sum = sum + value;
}
printf("%d", sum);
```



Thank You